2.3、性能测试

[OpenCV: Performance Measurement and Improvement Techniques]
https://docs.opencv.org/4.x/dc/d71/tutorial_py_optimization.html

这个教程的核心目标是：让你的图像处理代码不但正确，还必须足够快。因为图像处理每秒要处理成千上万的像素，速度非常关键

1. 性能测量（Measuring Performance）

OpenCV 提供了两个非常精确的计时函数：

- cv.getTickCount()：返回从某个参考事件（比如开机）到现在经过的 CPU 时钟周期数。
- cv.getTickFrequency()：返回每秒的时钟周期数（频率）。
  计算执行时间的方法非常简单：

```
import cv2 as cv

e1 = cv.getTickCount()   # 开始计时

# ======== 这里放你要测试的代码 ========

e2 = cv.getTickCount()   # 结束计时

time = (e2 - e1) / cv.getTickFrequency()   # 单位：秒
print(f"执行时间: {time:.4f} 秒")
```

2. OpenCV 默认就开启了优化（Default Optimization）

OpenCV 很多函数内部都用 SIMD 指令（SSE2、AVX 等）做了优化，几乎所有现代 CPU 都支持。

- 默认就是开启的。
- 你可以用下面两个函数检查和切换：

```
print(cv.useOptimized())        # 通常返回 True

# 禁用优化看看速度差别
cv.setUseOptimized(False)
print(cv.useOptimized())        # False
```

```
# 再启用回来（推荐一直保持开启）
cv.setUseOptimized(True)
```

结论：永远保持 cv.setUseOptimized(True)，除非你在调试。

3. 在 IPython / Jupyter 中更方便的计时工具

推荐使用 %timeit 魔法命令，它会自动多次运行取平均值，结果更准确。
示例1：Python 标量运算 vs NumPy

```
x = 5
%timeit y = x**2          # ≈ 73 ns
%timeit y = x * x         # ≈ 58 ns （更快！）

import numpy as np
z = np.uint8([5])
%timeit y = z * z                # ≈ 1.25 µs
%timeit y = np.square(z)         # ≈ 1.16 µs
```

重要发现：

- 小数据（单个或几个元素）时，纯 Python 标量运算远比 NumPy 快（可快 20~100 倍）。
- 数据量一大，NumPy/OpenCV 的向量运算就占优势。
  示例2：OpenCV vs NumPy 函数对比

```
%timeit z = cv.countNonZero(img)      # ≈ 15.8 µs
%timeit z = np.count_nonzero(img)     # ≈ 370 µs （慢了约 25 倍！）
```

结论：同等功能优先使用 OpenCV 函数，它们通常比 NumPy 更快（因为底层是高度优化的 C++）。

4. 性能优化的核心技巧（最重要部分）

- 尽量避免 Python 的 for 循环（尤其是双重、三重循环），它们非常慢。
- 最大程度向量化（Vectorize）：用 NumPy 的数组运算、OpenCV 的函数代替循环。
- 避免不必要的数组复制：尽量使用 view（视图）而不是 copy。
  - 例如：img[100:200, 100:200] 是 view，修改会影响原图；img[100:200, 100:200].copy() 才是复制。